

Scrutinize

Multi-Modal AI Embedding & Retrieval System Architecture & Project Guide

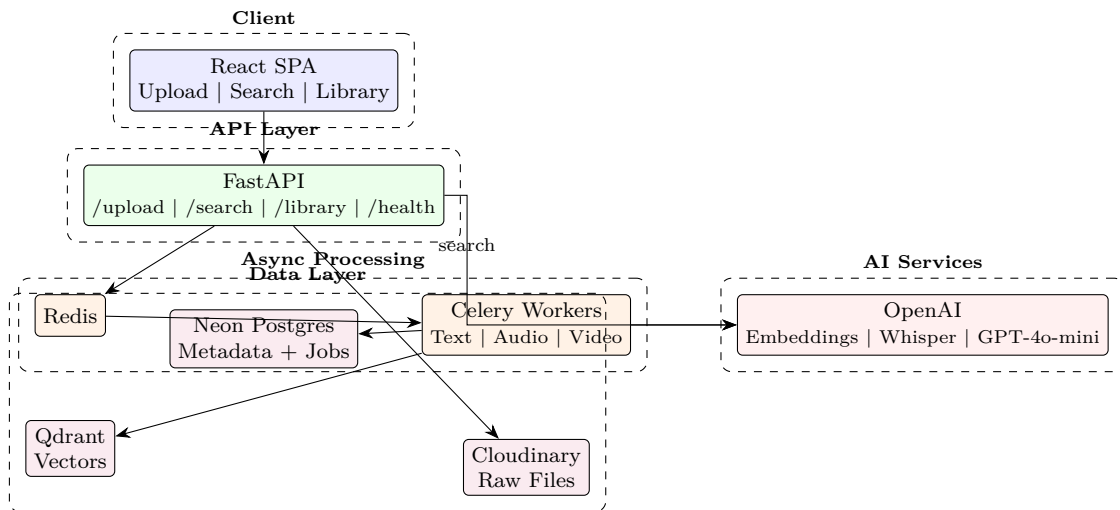
1 Overview

Scrutinize is a unified ingestion and retrieval system that lets users upload **text, audio, and video** files and later search across all modalities with a single natural-language query. A user might ask *“Find the video where someone drinks milk”* and receive a cited answer pointing to the exact timestamp in an indexed video—or match a lyric in an audio file, or a passage in a PDF.

The system follows a classic **Agentic RAG** pattern: raw files are processed asynchronously into searchable *segments* (text chunks with optional timestamps), stored as vector embeddings in Qdrant, and queried through a two-agent pipeline (router → retrieval → synthesis) that returns human-readable, source-cited answers.

2 High-Level Architecture

Four layers compose the system; a thin agent layer sits on top at query time.



Ingestion flow: the frontend uploads a file to `/upload`. FastAPI stores the raw binary in Cloudinary, creates `files` and `processing_jobs` rows in Neon, and enqueues a Celery task. The appropriate worker (text/audio/video) transcribes, captions, chunks, embeds, and upserts segments into Qdrant while mirroring metadata in Postgres.

Search flow: the user submits a query to `/search`. A **Router Agent** (GPT-4o-mini) decides modality filters and rewrites the query. The rewritten text is embedded, Qdrant returns top-*k* segments, and a **Synthesis Agent** produces a cited natural-language answer plus structured source cards for the UI (text snippets, audio/video players with seek).

3 Technology Stack

Embedding strategy (MVP): all modalities are converted to **text** (transcripts, captions, chunks) and embedded with `text-embedding-3-small` (1536-dim) into a single Qdrant collection. This enables cross-modal

Layer	Technology	Role
Frontend	React, Vite, Tailwind	Chat-style UI, upload, library browser
API	FastAPI, SQLAlchemy, Pydantic	REST endpoints, validation, OpenAPI docs
Queue	Redis + Celery	Background media processing
Vector DB	Qdrant	Similarity search with payload filters
Relational DB	Neon (Postgres)	Files, jobs, segments (source of truth)
Object Storage	Cloudinary	CDN-backed raw file URLs for playback
AI	OpenAI API	Embeddings, Whisper, vision captioning, agents
Media	FFmpeg	Audio extraction, keyframe sampling from video

search in one vector space. A planned Phase 2 adds named vectors (CLIP visual, CLAP audio) per Qdrant point for native visual/acoustic similarity.

4 Ingestion Pipelines

Text — Upload → Cloudinary → extract & chunk (~300–500 tokens) → embed → Qdrant + Neon.

Audio — Upload → Cloudinary → Whisper transcription (timestamped) → segment into 15–30 s windows → embed → Qdrant + Neon.

Video — Upload → Cloudinary → FFmpeg extracts audio track *and* keyframes → Whisper transcribes audio, GPT-4o-mini captions frames → merge into time-aligned segments → embed → Qdrant + Neon.

Each segment stored in Qdrant carries payload fields: `modality`, `content`, `file_id`, `start_time/end_time`, `source_path`, and `title`.

5 API Endpoints

Method	Path	Purpose
POST	<code>/upload</code>	Accept file; store in Cloudinary; enqueue processing job
GET	<code>/status/{job_id}</code>	Poll job progress (pending → indexed/failed)
POST	<code>/search</code>	Natural-language query → cited answer + sources
GET	<code>/library</code>	Browse indexed files and their segments
GET	<code>/health</code>	Dependency health (Neon, Redis, Qdrant, Cloudinary)

Rate limiting middleware protects upload and search endpoints. API keys (OpenAI, Cloudinary, Neon) remain server-side only; the frontend communicates exclusively with FastAPI.

6 Project Structure

```
Scrutinize/
  frontend/          React + Vite SPA (src/components, api/)
  backend/
    app/
      api/           upload, search, library routes
      services/      processors, embedding, vector store, agents
      workers/       Celery tasks (process_text/audio/video)
      models/        SQLAlchemy ORM (File, Segment, ProcessingJob)
      middleware/    rate limiting
      migrations/    Neon schema SQL
    tests/           unit, integration, system, security tiers
    docs/            architecture, modules, runbooks
    deploy/fly/      Fly.io deployment configs
    docker-compose.yml Redis, Qdrant, backend, worker
```

7 Getting Started

Prerequisites: Neon Postgres (pooled connection string), Cloudinary account, OpenAI API key, Docker.

1. Copy `.env.example` to `backend/.env` and fill in `DATABASE_URL`, `CLOUDINARY_*`, `OPENAI_API_KEY`.
2. Apply schema: `make install-backend` && `make db-migrate`
3. Start backend stack: `docker compose up -build` (Redis, Qdrant, API, Celery worker)
4. Start frontend: `make install-frontend` && `make frontend-dev`
5. Open `http://localhost:5173` (frontend) and `http://localhost:8000/docs` (API)

The frontend displays a green **API connected** badge when `/health` reports all dependencies are reachable.

Testing: `make test-unit`, `make test-integration`. CI (GitHub Actions) runs unit, integration, system, and security tiers on every push to `main`.

Deployment: production configs live under `deploy/fly/` for Fly.io (API, worker, Redis, Qdrant). See `docs/runbooks/fly-io-deploy.md` for the full runbook.

8 Design Decisions

- **FastAPI over Node gateway** — the ML/media stack (Whisper, FFmpeg, OpenAI SDK) is Python-first; a single-language backend avoids microservice overhead.
- **Qdrant over pgvector** — native vector DB with rich payload filtering and future multi-vector support per point.
- **Neon + Cloudinary split** — Postgres handles relational queries (jobs, joins, pagination); Cloudinary serves playable media URLs with CDN delivery.
- **Celery over BackgroundTasks** — video ingestion (frame extraction + multiple API calls) exceeds HTTP timeouts; workers scale horizontally.
- **Two-agent RAG** — Router Agent handles query understanding and modality filtering; Synthesis Agent produces cited answers. Minimal agent count, production-grade pattern.